POLYMORPHISM

Theory

- Definition: it comes from Greek and means "having many forms". It's one of the three pillars of OOP, along with encapsulation and inheritance.
- Main idea: in client code, we can treat related objects as the same kind of object. At runtime, each object does the correct task.
- The problem it solves: using instanceof with if/else to check the type of each object:
  - the code works, but it's a maintenance nightmare, because every new kind of object means adding another if;
  - the if/else statements are asking for errors, and the code is very verbose;
  - instead of telling objects to do things, we ask what they are and then tell them how to do it.
- The 3 steps to fix it:

1. Establish a common interface, for example an abstract method in the superclass
2. Implement that method in each subclass with @Override
3. Make the client code depend on the superclass.

- Consequences:
  - Client code is shorter and cleaner. Client code is any code that uses our objects, and all code is client code of something.
  - But we won't know what the client code will do until runtime, because it depends on the actual class of the object. So we trade one complexity for another.
- Extensibility: polymorphism generally improves it, because new behavior can be added in subclasses while the client code stays the same.
- Methods in Java:
  - Overriding: a new definition of an existing method, through inheritance.
  - Overloading: a new definition of a method with the same name in the same class.

Scanned with CamScanner

- David Wheeler (around 1972): "All problems in computer science can be solved by another level of indirection, except of course for the problem of too many indirections". It means that adding another layer of abstraction can solve almost any problem, but too many layers become a problem too.
- Example: chess pieces.

```
abstract class ChessPiece {
    public abstract Location[] getPossibleMoves();
}
class Pawn extends ChessPiece {
    @Override
    public Location[] getPossibleMoves() { /* pawn moves */ }
}
class Rook extends ChessPiece {
    @Override
    public Location[] getPossibleMoves() { /* rook moves */ }
}
// Client code
for (ChessPiece piece : listOfChessPieces) {
    Location[] possibleMoves = piece.getPossibleMoves();
}
```

ABSTRACTION

Theory

- Definition: abstractions means considering only the relevant parts of an objects and ignoring the others. It means summarizing or generalizing something to focus on what matters for a specific purpose.
- As a process (verb): bringing certain details to the front while hiding everything else. In programming, it's creating an interface for a component, like a class definition, that shows the details needed to work with it and hides the rest.
- As a result (noun): a description or definition that leaves out unnecessary details. A class definition is an abstraction.
  - Public properties and methods are the exposed details.
  - Private properties and the method implementations hide details that users don't need.
  - Irrelevant details are left out completely.
- John Guttag: abstraction is about keeping the information that's relevant in a given context and forgetting the information that isn't.
- Abbott et al.: an abstraction is the "reification and conceptualization" of an distinction".
- Creating good abstractions is hard, because developers have to sift through a lot of details to find the most important concepts.
- Levels of abstraction: a class with fewer details is more abstract. For example, ChessPiece is more abstract than Pawn.
- layer of abstraction: we find the similarities between systems and put them in a layer of abstraction, and the specific differences go to concrete subclasses. This layer can't stand on its own, which is why it's abstract.
- Two common problems:
  - leaky abstraction: other components end up relying on details that aren't stated in the abstraction. Good encapsulation can often solve this.

Scanned with CamScanner

- Over abstraction: too much is "forgotten", so users don't get enough control or access to the information.

- Abstraction principle: every component should clearly declare its accesible features, no more and no less than what other components need. Following it improves understandability, testability, maintainability and reusability. It also helps with modularity, because it shows problems with localization of design decisions, unnecessary coupling and low cohesion.

- Adherence criteria:

1. Meaningful labels and identifiers: names should be precise, not too broad or too narrow. Classes and variables should be nouns, and methods should be verbs.
2. Context-aware labels and identifiers: names should make sense in the context where they're declared and used.
3. Explicit declaration of the full abstraction: everything that's exposed, whether on purpose or by accident, should be in the class definition or in the documentation.
4. Sufficiency (completeness): users of a class should be able to do everything they need with its objects.
5. Conciseness (non-redundancy): there shouldn't be repeated mechanisms for the same thing, unless they're needed for legacy code or an adapter.

- Example:
  - Attributes: id, breed, banOn, isNothing
  - Methods: getters / setters and getAgeInMonths()

INTERFACES

Theory

- Abstract class: you can't create (instantiate) an object of an abstract class.
- Abstract method: the derived class must implement it.
- Interface:
  - an interface can inherit from another interface;
  - a class implements an interface.
- Public interface of class: all of its public elements, such as data members, methods and inner classes.
- Interfaces can be used as reusable definitions for some or all of a class's public interface.
- Things a class definition can't capture: for example the order in which methods must be called or constraints on specializations.
- Java keywords: abstract, interface, implements, @Override
- Example:

```
interface IFarmAnimal { ... }
IFarmAnimal farmAnimal = new FarmAnimal();
farmAnimal = new Chicken();
```

Scanned with CamScanner